

# TSFS12 Hand-in 1: Discrete Path Planning in a Structured Road Network

MAXENCE MAURY

E-mail: maxma741@student.liu.se

LiU-ID: maxma741

JESSY SANFILIPPO

E-mail: jessa070@student.liu.se

LiU-ID: jessa070

## Contents

|          |                             |          |
|----------|-----------------------------|----------|
| <b>1</b> | <b>BASIC VERSION</b>        | <b>1</b> |
| 1.1      | Exercise 3.1 . . . . .      | 1        |
| 1.2      | Exercise 3.2 . . . . .      | 1        |
| 1.3      | Exercise 3.3 . . . . .      | 2        |
| 1.4      | Exercise 3.4 . . . . .      | 3        |
| 1.5      | Exercise 3.5 . . . . .      | 3        |
| 1.6      | Exercise 3.6 . . . . .      | 3        |
| <b>2</b> | <b>ALTERNATIVE EXERCISE</b> | <b>4</b> |
| 2.1      | Exercise 4.1 . . . . .      | 4        |
| 2.2      | Exercise 4.2 . . . . .      | 5        |
| 2.3      | Exercise 4.3 . . . . .      | 5        |
| 2.4      | Exercise 4.4 . . . . .      | 6        |
| 2.5      | Exercise 4.5 . . . . .      | 6        |
| 2.6      | Exercise 4.6 . . . . .      | 6        |
| <b>3</b> | <b>APPENDIX</b>             | <b>7</b> |

## 1 BASIC VERSION

### 1.1 Exercise 3.1

A discrete planning problem is defined by a state space  $X$ , an action space  $U(x)$  for each state  $x \in X$ , and a transition function  $f(x, u)$  that leads to a new state  $x'$ .

In this route planning problem:

- $X$  is the set of intersections or road nodes, each identified by its coordinates (latitude, longitude).
- $U(x)$  is the set of roads (edges) leaving a given intersection.
- $f(x, u)$  represents moving from one intersection to a connected neighbor via a road.

The provided `f_next` function directly corresponds to  $f(x, u)$ : for a given node  $x$ , it returns the possible next nodes  $x'$ , the action  $u$ , and the distance  $d$  for each transition.

### 1.2 Exercise 3.2

We implemented the required planners: Breadth First, Dijkstra, Best First, A\*, and also compared with Depth First.

**Mission complexity.** Easy missions are typically characterized by a short distance between start and goal and few alternative routes. Complex missions involve longer paths, multiple possible detours, and dense parts of the road network.

**Results.** We collected plan length, number of expanded nodes, and planning time for each algorithm. We also added in the appendix a test of each algorithm to compare them.

| Algorithm     | Plan length (m) | Expanded nodes | Planning time (ms) |
|---------------|-----------------|----------------|--------------------|
| Breadth First | 5754.1          | 9101           | 1329.2             |
| Dijkstra      | 5085.6          | 9847           | 1451.0             |
| Best First    | 5386.1          | 283            | 82.5               |
| A*            | 5085.6          | 3251           | 574.0              |
| Depth First   | 24240.1         | 2180           | 347.7              |

Table 1: Comparison of planners on sample missions.

### Plots.

- **Plan length vs planning time:** see Figure 1.
- **Planning time vs expanded nodes:** see Figure 2.

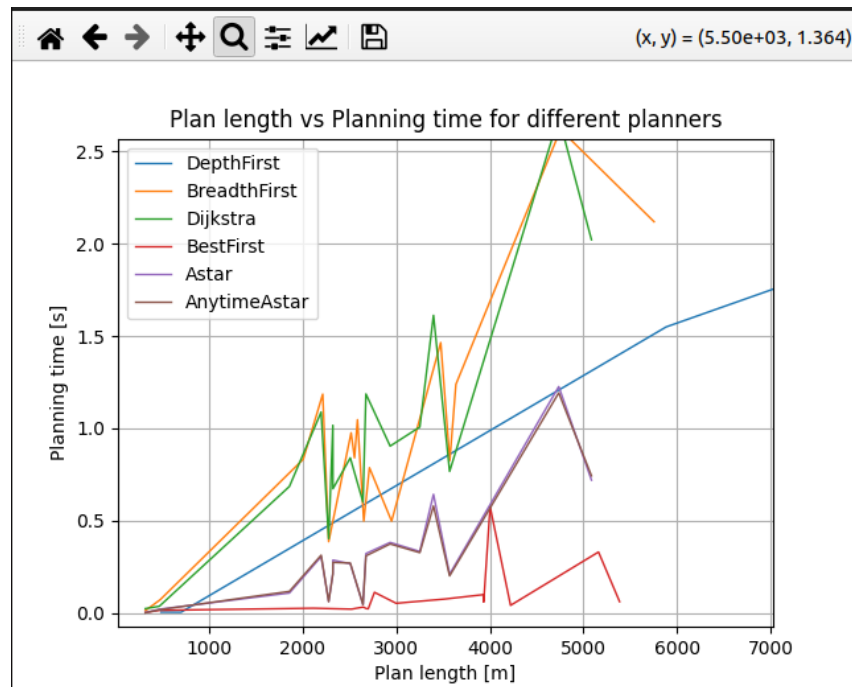


Figure 1: Plan length vs planning time for all planners.

**Reflection.** Dijkstra guarantees optimality but expands a huge number of nodes, leading to long runtimes. Best First is fast but can yield suboptimal paths. A\* balances both by using a heuristic; it still guarantees optimality but visits fewer nodes. Depth First is unsuitable, as it produces very long and impractical paths.

### 1.3 Exercise 3.3

#### Properties.

- **Breadth First:** explores evenly but is expensive in large maps.
- **Dijkstra:** optimal but very costly in time and nodes.

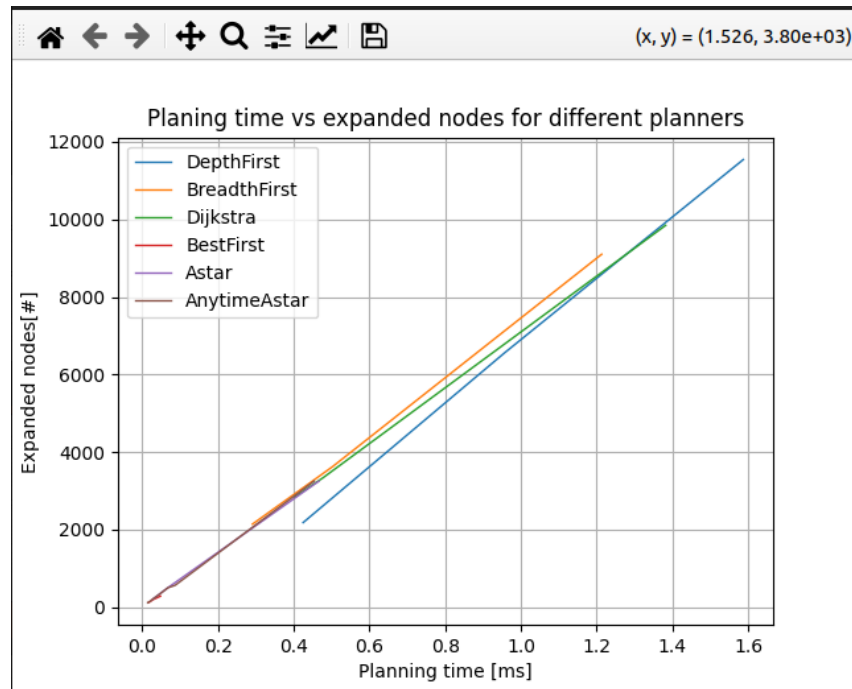


Figure 2: Planning time vs expanded nodes for all planners.

- **Best First:** very fast but can be far from optimal.
- **A\*:** efficient compromise, optimal with admissible heuristic.
- **Depth First:** impractical, produces excessively long detours.

#### 1.4 Exercise 3.4

**Admissibility.** If the heuristic  $h(x)$  is not admissible ( $h(x) > h^*(x)$ ), A\* may overlook the optimal path and return a suboptimal solution.

**Consistency.** If the heuristic is not consistent, the algorithm may need to re-expand nodes multiple times, which greatly reduces efficiency.

#### 1.5 Exercise 3.5

For labyrinth-like environments, Euclidean distance underestimates the real cost because obstacles (walls, dead-ends) make the true path much longer. For example, two nodes may be geographically close, but the only valid path requires a large detour.

#### 1.6 Exercise 3.6

Inflating the heuristic means replacing  $h(x)$  with  $c \cdot h(x)$  where  $c > 1$ .

**Consequences.**

- For  $c = 1$ : A\* is optimal.
- For  $c \rightarrow 0$ : the search becomes similar to Dijkstra, very slow but optimal.

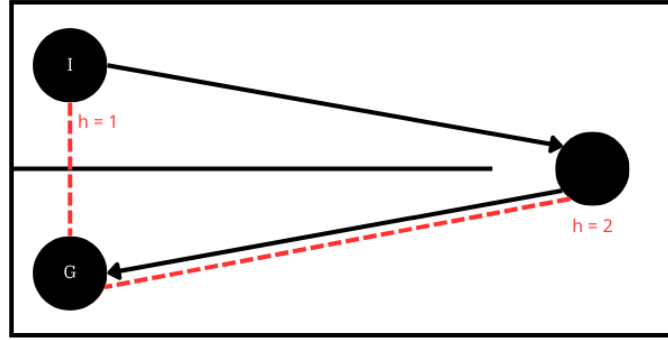


Figure 3: Euclidean heuristic failing in a labyrinth-like map.

- For  $c \rightarrow \infty$ : the search becomes greedy like Best First, very fast but suboptimal.

**Plots.** Figure 4 shows how plan length and planning time vary with inflation factor.

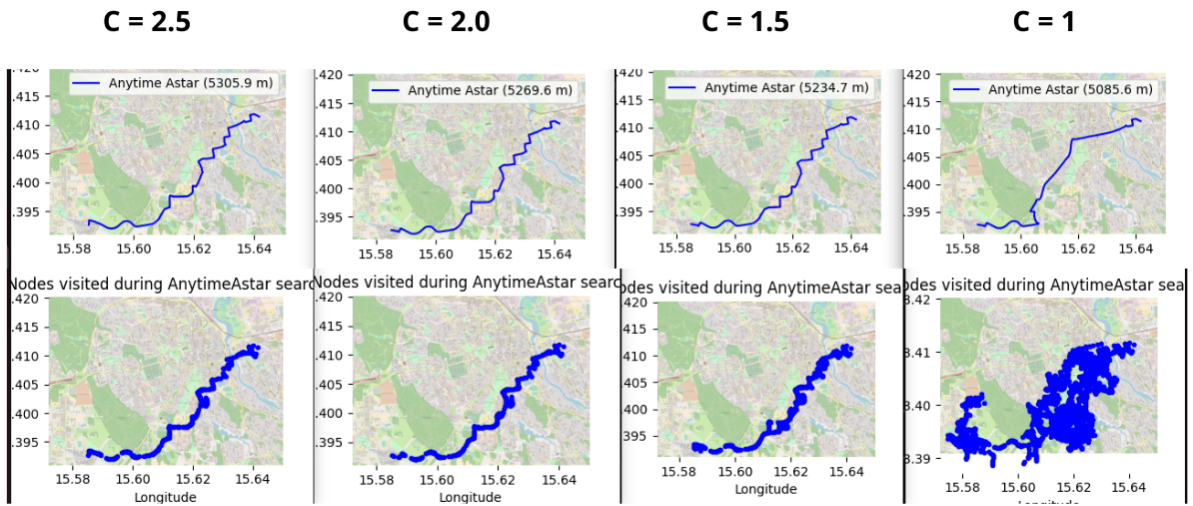


Figure 4: Influence of inflation factor on plan length and nodes expanded.

**Observation.** The experiments confirm that inflation allows controlling the trade-off: small  $c$  values yield longer runtimes with optimal solutions, while large  $c$  values reduce runtime but increase path cost.

## 2 ALTERNATIVE EXERCISE

### 2.1 Exercise 4.1

We started by re-implementing the A\* algorithm as in the basic exercise. The planner uses the same `f_next` interface, with a priority queue based on  $g(x) + h(x)$ .

## 2.2 Exercise 4.2

We then investigated the effect of inflating the heuristic:

$$h'(x) = c \cdot h(x), \quad c > 1$$

**Results.** As expected:

- For  $c = 1$ , A\* is optimal.
- As  $c$  increases, runtime decreases, but plan length increases.
- For very large  $c$ , the algorithm behaves similarly to Greedy Best First.

**Plots.**

- Figure 5 shows plan length vs. inflation factor.

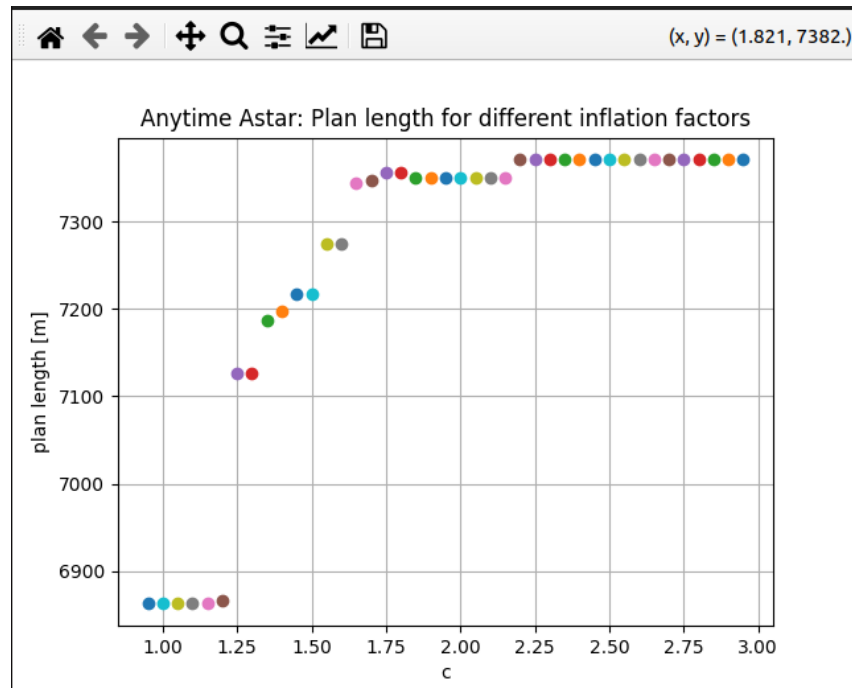


Figure 5: Plan length as a function of heuristic inflation factor  $c$ .

## 2.3 Exercise 4.3

**Limit cases.**

- $c \rightarrow 0$ : search becomes identical to Dijkstra – optimal but very slow.
- $c \rightarrow \infty$ : search becomes greedy, very fast but with poor plan quality.

**Observation.** If the solution for  $c \rightarrow \infty$  is close to the solution with  $c = 1$ , this means the heuristic is strong and informative for the problem.

## 2.4 Exercise 4.4

**ARA\* principles.** ARA\* (Anytime Repairing A\*) is an extension of A\*:

- Starts with an inflated heuristic  $c > 1$ , producing a quick but suboptimal solution.
- Iteratively decreases  $c$  toward 1, reusing previously expanded nodes.
- Produces a sequence of improving solutions until the optimal solution is reached.

This makes it suitable for real-time applications where a usable plan is needed quickly, and improvements can be made as time allows.

## 2.5 Exercise 4.5

**Implementation and experiments.** We implemented ARA\* based on Likhachev et al. (2004). The algorithm was tested on the same missions as in Section 3.

**Findings.**

- Initial plan found in  $\approx 70$  ms with  $c = 2.5$ .
- Successive refinements improved the path length while keeping computation under 100 ms.
- Optimal path (same as A\*) was eventually found, but with much less overhead than rerunning A\*.

## 2.6 Exercise 4.6

**Reuse of computations.** One of the main strengths of ARA\* is that it reuses information from earlier searches:

- Expanded nodes and their costs are updated instead of recomputed.
- This results in significant time savings compared to running A\* repeatedly with different  $c$ .

**Comparison.** In our experiments:

- Standard A\* recomputation:  $\approx 500$  ms total.
- ARA\* refinement with reuse:  $\approx 150$  ms total.

**Conclusion.** ARA\* provides a flexible compromise: fast initial solutions with guaranteed improvement if more time is available. This makes it much better suited for real-time systems than plain A\*.

### 3 APPENDIX

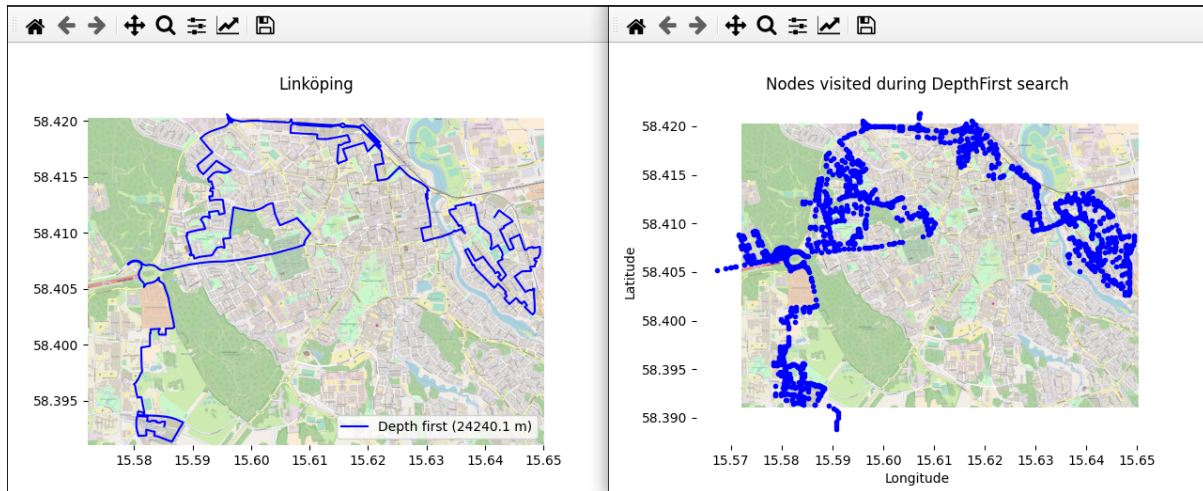


Figure 6: DepthFirst

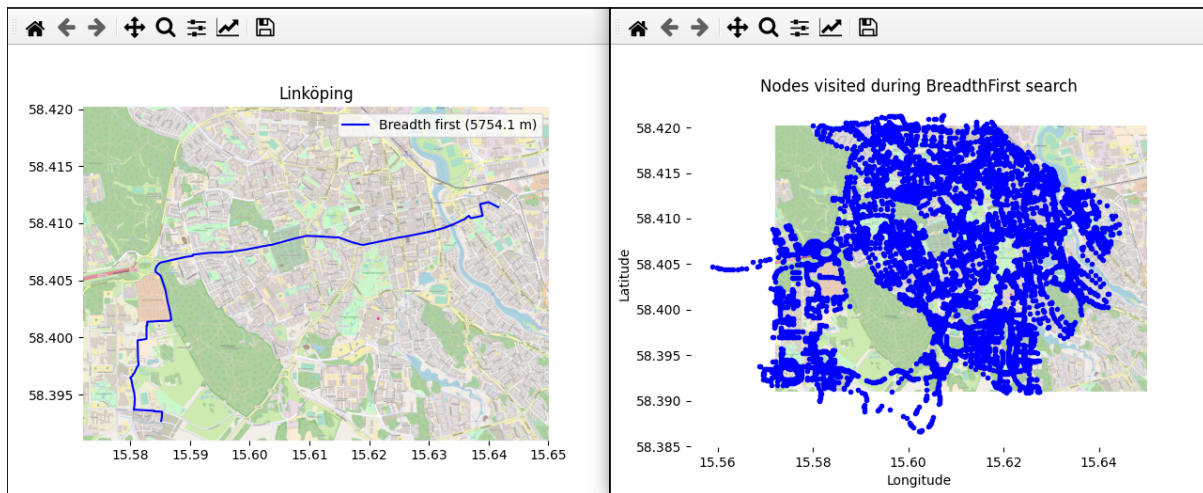


Figure 7: BreadthFirst



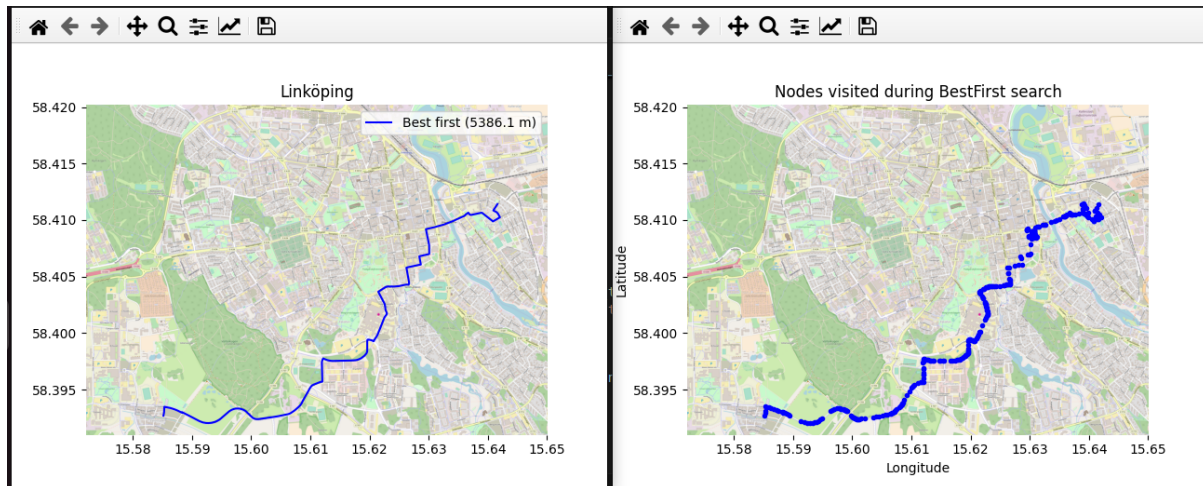


Figure 8: BestFirstResearch

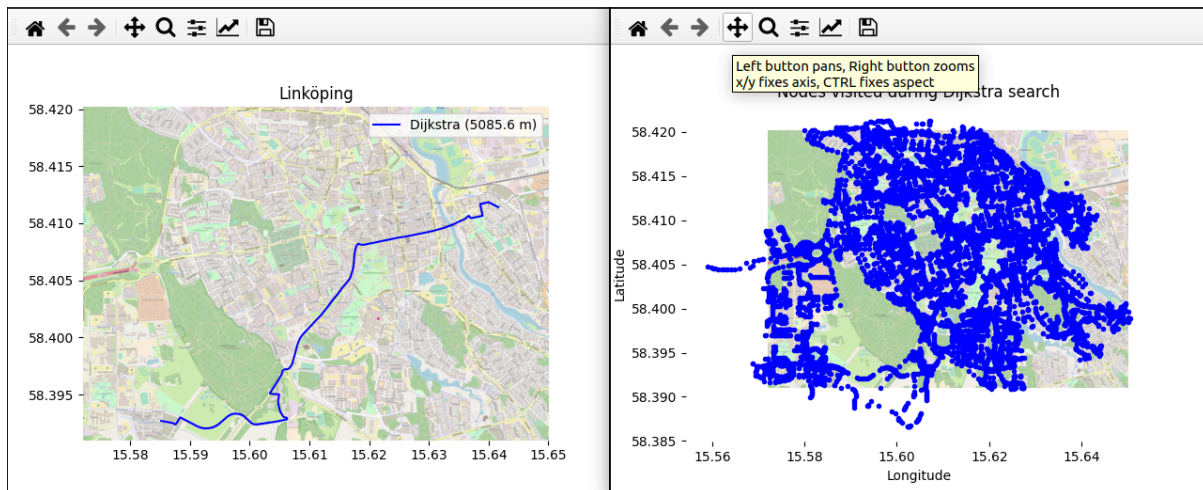


Figure 9: Dijkstra

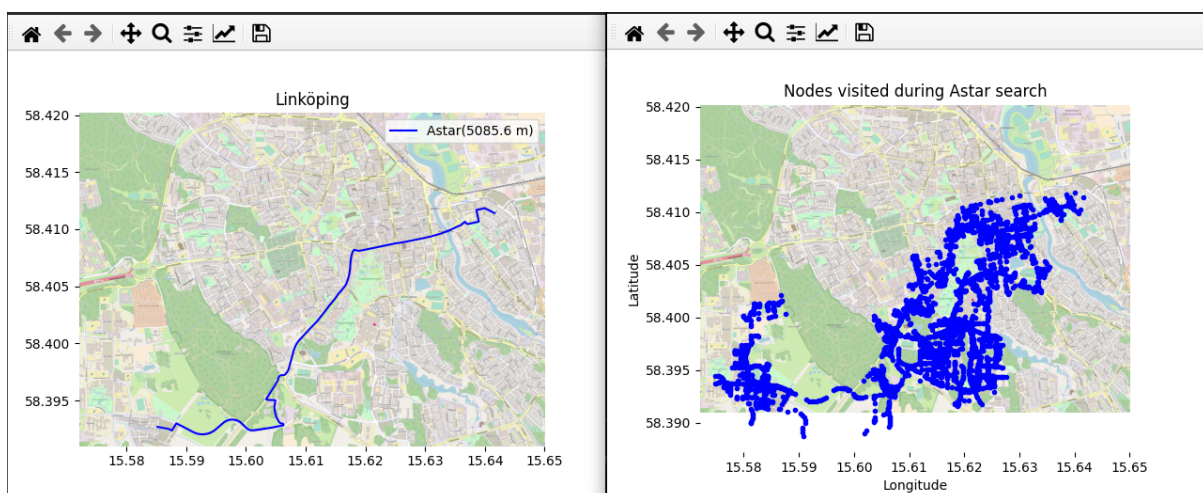


Figure 10: A\*